Do not panic on seeing the *INSERT* statement messages—they simply convey that zero records were inserted in the base (master) table. Actually, the records were transparently inserted into the related partitions. Another thing to note about the *UPDATE* statement is that you are not supposed to update the partitioning key. Basically, any change to the partitioning key value might result in the movement of that record to another partition, which is termed as the row movement. A simple way to handle queries that do update the partitioning key is to capture the *UPDATE* query, delete the related record from the partition, and then fire an *INSERT* on the base table. Then the partitioning mechanism will kick in and redirect the record to the appropriate partition. This is not handled in the current set-up, as I am updating the address value, which is a non-partition-key column.

Let's now check where the records are stored:

```
SELECT * FROM orders;
 id |  address   |     order_date
----+------------+------------------------
  1 | pune       | 22-AUG-11 00:00:00
  2 | bengaluru | 22-FEB-10 00:00:00
(2 rows)
```

PostgreSQL knows about the child tables of the *orders* table, so it assumes that the user wants all the data, from the parent as well as all the children. The partitions are normal tables, and you can query them as usual:

```
SELECT * FROM orders_part_2011;
 id | address |     order_date
----+---------+---------------------
  1 | pune    | 22-AUG-11 00:00:00
(1 row)

SELECT * FROM orders_part_2010;
 id |  address  |     order_date
----+-----------+---------------------
  2 | bengaluru | 22-FEB-10 00:00:00
(1 row)
```

You can also check if the master table is really empty, by using the *ONLY* clause, which restricts the look-up to only the table specified in the statement:

```
SELECT * FROM ONLY orders;
 id | address | order_date
----+---------+------------
(0 rows)
```

Querying over partitions

Use the *EXPLAIN* feature to check the plan for querying over partitions:

```
EXPLAIN SELECT * FROM orders WHERE order_date = '02-JAN-11';
                                QUERY PLAN
-------------------------------------------------------------------------
 Result  (cost=0.00..26.01 rows=7 width=40)
   ->  Append  (cost=0.00..26.01 rows=7 width=40)
         ->  Seq Scan on orders  (cost=0.00..23.75 rows=6
width=44)
               Filter: (order_date = '02-JAN-11
00:00:00'::timestamp without time zone)
         ->  Seq Scan on orders_part_2011 orders
(cost=0.00..2.26 rows=1 width=18)
               Filter: (order_date = '02-JAN-11
00:00:00'::timestamp without time zone)
(6 rows)
```

In the above output, you see that only one partition was scanned, based on the *WHERE* clause conditions. Let's look at another example:

```
EXPLAIN SELECT * FROM orders WHERE order_date = now();
                                QUERY PLAN
-------------------------------------------------------------------------
 Result  (cost=0.00..30.03 rows=8 width=41)
   ->  Append  (cost=0.00..30.03 rows=8 width=41)
         ->  Seq Scan on orders  (cost=0.00..26.50 rows=6
width=44)
               Filter: (order_date = now())
         ->  Seq Scan on orders_part_2011 orders
(cost=0.00..2.51 rows=1 width=18)
               Filter: (order_date = now())
         ->  Seq Scan on orders_part_2010 orders
(cost=0.00..1.01 rows=1 width=44)
               Filter: (order_date = now())
(8 rows)
```

Here all the partitions are scanned--definitely not what we wanted! You should be aware of the fact that the planner analyses the query before the values from the parameters or stored procedures are substituted. As the planner does not know the exact value of now() during the planning phase, it cannot prune partitions, and so scans all the partitions. You need to look out for such cases where constant values are expected. In case you are planning to use functions in the *WHERE* clause, do make sure to understand the various types of functions that can be created in PostgreSQL.

Let us now go through the finer details of the three features used in PostgreSQL partitioning.

Constraint exclusion

Constraint exclusion works with only range or equality check constraints. It might not work for constraints like the following: